

SPTCal deployment-readiness audit

24 Sep 2026. Audited: the live build at `a432613` (`dist/index.html`, SHA-256 `3630896c...`, 1,281,946 bytes; the last full gate passed 376/0). Nothing in `src/`, `index.html`, `tests/` or the baselines was changed. All test pages, scripts and hostile files were kept in the session scratchpad. Nothing is committed.

At a glance

- **Verdict: NO-GO as the build stands. GO once the six must-fix items below land and the gate is green.** None of them needs a frozen edit. Two need your ruling, because they change what the exports contain.
- **4 high, 15 medium, 26 low, 10 notes.** There are also 13 suspicions I could not reproduce, listed separately.
- **The biggest risks:**
 1. **Security:** a crafted calendar or header-preset file runs script when opened.
 2. **A broken feature:** every "Export shareable copy" made since the Mantine cutover opens broken.
 3. **Wrong dates:** any phase with "Snap to Mon" off is placed a week late in the waterfall, Excel and waterfall PDF, and a month-view pill drag turns snap off automatically.
 4. **Corrupted files:** a file that fails partway through loading leaves a mix of two calendars on screen, and the next Save writes that mix into the user's own file.
- **Month PDF:** the direct writer does **not** need to ship before deployment. At default print settings the print path is sound. §8 has the print-dialog measurements and §9 the "writer must replicate" list.

1. How this was tested

- **Phase 1 (debug):**
 - Eight read-only tracer agents, one per surface. They read the code and wrote 86 harness scripts and 36 hostile fixtures.
 - Every script then ran through **one serial execution lane:** a copy of `run.sh` and `srv.js` in the scratchpad, with its own port (8391) and Chrome profile. It never shared output files with the repo's harness.
 - Every critical and high finding then went to an adversarial skeptic who tried to refute it (14 skeptics). Medium and low findings needed a clean reproduction.
- **Real browser:** the built-in pane against `dist/` served by `cal-dist-5175`. It was used for layout, the proof that Save writes a mixed calendar, crash recovery, scale timing and the shareable copy.
- **Print dialog:** Chrome's dialog options were driven through CDP `Page.printToPDF` on the real month print document, and the pages were rendered under Quartz (`pdfseparate + sips`), not poppler.
- **The probe build:** most scripts ran against `/probe.html`, the engine built unminified from HEAD. Its only addition is `window.__SPT`, a frozen table of 24 named engine functions (no string evaluation). An `eval` hook was my first idea, and the auto-mode safety check blocked it as an RCE surface, so I dropped it.
- **The real artifact:** every high and medium finding that shows on screen or in an export was re-run or re-proved on the untouched `dist/index.html`. Where that was not done, the entry says so.
- **A harness trap found along the way:** Chrome's `--dump-dom` exits with a 0-byte dump when the DOM holds a lone UTF-16 surrogate. A control on a bare page proved this is Chrome and not the app. Scrub such characters before dumping. It will be recorded in HANDOFF.

Each entry below gives what happens, the repro steps, the evidence, the suggested fix, whether it needs your ruling (frozen surface / save format / exports), and a rough size. Evidence paths are relative to the session

scratchpad (\$SP = /private/tmp/claude-510/-Users-apple-Desktop-Calendar-Builder/955ba3ba-ac7a-438b-9970-d1b7c7df17d8/scratchpad).

2. High

H-1. A crafted calendar or header-preset file runs script when opened (stored XSS)

- **What happens:** values read from a `.sptcal`, a legacy `.html`, a shareable copy's inline state or an imported `.spthdr` preset are pasted **unescaped** into `innerHTML` attributes:
 - header-line formats (colour, highlight, size, alignment) go into `style=""`;
 - note, hiatus and day-note colours go into `style`;
 - a custom holiday's `id` goes into `data-hid=""`;
 - episode and block ids and day counts, and hiatus start and weeks, go into the sidebar rows.

A value like `#FF0000;"><img src=x onerror=...>` breaks out and runs. The payload is ordinary state, so it rides into every re-save, autosave, shareable copy, undo step and crash backup. One poisoned file spreads as people pass it on.

Text the user **types** is escaped correctly. No typed-text XSS was found.

- **Why it matters here:** the script runs on the shared `greicher1.github.io` origin. There it can read the crash backup and the recent-file handles in IndexedDB, rewrite the open calendar and let autosave write it, and read other Pages apps' storage on that origin. There is no CSP to limit it.
- **Repro:**
 1. Take any real `.sptcal` and set `headerFormat.c1.color` to `#FF0000;" data-x="1"><img src=x onerror="console.log('xss')">`.
 2. In the app, choose File ▶ Load... and pick the file. The marker fires as the grid renders, in Auto header mode as well as Manual.
 3. Or: import a `.spthdr` preset carrying the same colour and apply it. It fires on every calendar the preset is applied to.
- **Evidence:**
 - On `dist.html`, the `XSS_stylebreak` runs (6 fixtures: header format, header format in Auto, note colour, hiatus colour, holiday id, day-note colour) and the `PROD_xss` / `PROD_xssblk` runs (episode and block ids and days, holiday id) all fired their inert markers.
 - The picker path and the inline path both fire (`SAVE_xss.picker`, `SAVE_xss.inline`).
 - The `.spthdr` import-and-apply path fires (`GRID_xsshdr`), and the payload persists in the snapshot (`snapshotCarriesPayload: true`).
 - Files: `$SP/out/runs/XSS_*.json`, `PROD_xss*.json`, `SAVE_xss*.json`, `GRID_xss*.json`, `SUPPLY_hostile.json`. The skeptic's verdict: confirmed, high.
- **Fix:** validate at the two import boundaries, which are **not frozen**, instead of escaping inside the frozen renderers (which would be a frozen edit):
 1. In `applyStateSnapshot`, and in the `.spthdr` reader, colours must match `/^[0-9a-f]{6}$/i`, sizes must be finite numbers in range, alignments must be from a fixed set, ids must match `/^[\\w-]+$/`, day counts must be numbers, and dates must be ISO. Anything else falls back to the default.
 2. Escape `addHiatusRow`'s `prefillStart` / `prefillWeeks` (see M-3).
 3. Add the proposed CSP (L-4) as defence in depth.
- **Needs a ruling:** no frozen edit and no save-format change. The values it rejects could never come from the UI. The gate must run (save path).

- **Size:** small-medium, about 50–70 lines plus hostile-file harness legs.

H-2. "Export shareable copy" opens broken: the whole chrome is rendered twice

- **What happens:** the exported `.html` opens with **two sets of header controls, two toolbars, twelve sidebar cards instead of six**, and duplicate ids (`show-title`, `export-btn`, `undo-btn`, `file-menu` and more). The page also scrolls sideways (1197 px wide in a 1024 px window). Only one of the two sets of controls is wired up. The cause is that `buildSavedHtml` clones the document with React's rendered chrome still inside it, and `main.jsx`'s `createRoot` renders a second copy on top when the file opens. **This affects every shareable copy exported since the 31 Aug cutover**, and every plain Save on a browser without the File System Access API. HANDOFF does not record it.
- **Repro:** Settings ▶ App ▶ Export App With Data. Open the downloaded `.html` by double-clicking it.
- **Evidence:**
 - Re-proved cleanly in the browser pane with no harness involved. A copy captured from `dist` at `localhost:5175` (1,438,562 bytes) was written in as the document: `showTitleIds: 2`, `exportBtns: 2`, `undoBtns: 2`, `docW: 1197`, `W: 1024`, with a screenshot.
 - The same result on a `file://` open via CDP (`$SP/out/share-copy-open.png`), and in `SUPPLY_copyopen.json`.
 - The skeptic's verdict: confirmed, high.
- **Fix:** in `buildSavedHtml`'s clone, empty the React hosts (`header.app-header`, `.view-toggle-row`, `#sidebar-static`, `#hiatus-hint-host`, `#react-root`) next to the existing `#table-wrap` / `#print-root` emptying. Optionally, clear each host in `main.jsx` before rendering. This also stops the sender's recent file names travelling in the copy (L-21).
- **Needs a ruling:** no frozen edit. The save format is untouched: only the `saved-state` block is ever read back. Gate required.
- **Size:** small (1–4 lines) plus a `sharecopy` leg that counts ids.

H-3. A phase with "Snap to Mon" off is placed a week late in the waterfall, Excel and waterfall PDF

- **What happens:** when a phase starts on a Tuesday–Sunday with snap off, its first (partial) week is left out. The phase appears to start the following Monday, and Production can gain a phantom week after wrap. The month view is correct, so the two views disagree. The gap banner also over-counts. **A month-view pill drag that lands on any non-Monday turns snap off by itself**, so ordinary use hits this.
- **Repro:**
 1. On the Phases tab, untick Snap to Mon on Pre Prep and type Wed 4/8/26. Or in Month view, drag the Pre Prep pill by a day.
 2. Switch to Waterfall. Pre Prep Wk 1 sits on the 4/13 row, and the 4/6 row is empty.
 3. The gap banner reads "3/30/26–4/12/26 (2 wk)" when the real gap is 1 week.
- **Evidence:**
 - `SCHED_snapgrid.json` on `dist.html`, with the real fixture `dayoverrides.sptcal`: rows 4/13...5/18, and the banner as quoted above.
 - `PROD_sched.json`: a Wednesday Production start shows first on 7/6, and a one-day Friday shoot vanishes from the grid entirely.
 - `SCHED_core2.json` P1.
 - The skeptic's verdict: confirmed, high. The skeptic also ran the fix against the real function: 400 random snapped calendars came out identical.

- **Fix:** in `computeSchedule` (not frozen), use an overlap test for week activity (`s.start < weekStart+7 && s.end > weekStart`), and compute Production's week span from `mondayOf(start)`.
- **Needs a ruling: yes, because the exports change.** Excel and waterfall-PDF output changes for snap-off calendars, which is the point. Snapped calendars (every gate fixture) are unaffected. Gate required.
- **Size:** small-medium, about 10–20 lines. Ship it with H-4.

H-4. When the first phase is snap-off, every waterfall row moves off Monday, and notes and hiatus names vanish

- **What happens:** if the earliest phase starts on a non-Monday with snap off, the grid's origin becomes that weekday. The Date column prints Fridays or Wednesdays. Every store keyed by Monday (notes, hiatus names and colours, row heights, cell spans) then matches no row, so **saved notes and named hiatuses disappear from the grid, Excel and the PDF**. A note typed in that state is orphaned again once snap is restored. Single Column Mode is always exposed. Full-year mode is exposed when the date falls before that year's first Monday.
- **Repro:** turn Single Column Mode on. In Month view, drag the Writer's Room pill one day. Switch to Waterfall: dates read 1/7/26, 1/14/26..., and existing notes are gone.
- **Evidence:** `SCHED_anchor.json` and `SCHED_anchorcol.json` on `dist.html`:
 - first rows dated Fri 1/2/26 in full-year mode and Wed 1/7/26 in one-column mode;
 - the note on 2/2/26 is not visible anywhere;
 - named bands show as "Hiatus";
 - the Production row is dated 7/3/26.

The skeptic's verdict: confirmed, high.

- **Fix:** in `computeSchedule`, snap the grid's geometry to Mondays (`naturalStarts` via `mondayOf`, and ends rounded up), keeping the day-accurate start for the month view.
- **Needs a ruling:** yes, because the exports change for affected calendars. Not frozen. Gate required.
- **Size:** small, 2–4 lines, together with H-3.

3. Medium

M-1. A load that fails partway leaves a mix of two calendars, and the next Save writes it into the user's own file

- **What happens:**
 1. If a file parses as JSON but throws inside `applyStateSnapshot` (a `null` field value, a `null` hiatus, a colour index this build doesn't know), the user sees "Could not load a file...".
 2. Behind that notice, the previous calendar has been **partly overwritten** by the bad file.
 3. `suppressDirty` stays stuck on. There is no "Unsaved changes", no crash backup, no undo steps, no autosave and no leave-page warning until the next undo, New or successful load.
 4. The file handle still points at the user's own file, so **their next Save writes the mix into it**.
- **Repro (proved in the browser pane with a real Save click):**
 1. Load `A.sptcal` (hiatus "A BREAK").
 2. Load `B.sptcal`, which carries a `null` entry in `fields.hiatuses`. The notice appears. The sidebar now shows "B BREAK", and "A BREAK" is gone.
 3. Click OK, then Save. **One write lands on `A.sptcal`: title "SHOW A" with hiatus "B BREAK"**. The status reads "Saved 5:50 PM".

4. Edits made after that showed no "Unsaved changes". On reload there was **no recovery prompt**, and the edits were gone. In the control session the same edit showed "Unsaved changes", and on reload "Recover unsaved work" restored it.
- **Evidence:** \$SP/findings.md §F2 and "F2 UPGRADE" (the pane transcript); MAIN_load.
{nullfield,future,control}.json; SAVE_throw.*.json; SAVE_throw2.*.json; SAVE_parse.json P2 (four throw sites).
 - **Why medium, not high:** the skeptic lowered it because no file the current app writes triggers it. It needs a hand-edited file, corruption that still parses, or a version mismatch. **One sanctioned future change creates the trigger:** appending a colour to PHASE_COLOR_OPTIONS. Any older engine (a shareable copy, a tab left open) would then throw on new files. Fix this before any palette append.
 - **Fix:**
 1. Make every restore all-or-nothing (openRecentFile, applySnapshotJSON, backup recovery, boot restore). Snapshot first; try { apply } catch { re-apply the previous state; rethrow } finally { suppressDirty = false }.
 2. Make the four throw sites fall back instead of throwing.
 3. Refuse version > SNAPSHOT_VERSION with a clear message.
 - **Needs a ruling:** no frozen edit and no format change. Gate required.
 - **Size:** small-medium, about 40–70 lines. It shares the boundary with H-1, so do the two together.

M-2. Loading calendar B after calendar A carries A's custom phases into B (known, but worse than recorded)

- **What happens:** applyStateSnapshot rebuilds custom phases only when the incoming list is **non-empty**, and loading never resets first. So A's custom phases, with their names, dates, colours and hiatus settings, stay in B's sidebar, grid, month view and exports. They also get written into B's file on the next save. Undoing past "Add phase" leaves an empty custom row behind by the same route.
- **Repro:** load a calendar with a custom phase (e.g. "Casting" 10/5/26 ×4), then load one with none, without choosing New first. "Casting wk 1–4" shows in B's grid.
- **Evidence:** MAIN_carry_dist.*.json (dist), carrydiff.*.json (whole-snapshot diff: the **only** 18 differing paths are the custom-phase ones, and every other store resets correctly, including against the v1.0.0 fixture), SAVE_carry_dist.json C3, SAVE_reset.json R2 (undo).
- **Known:** HANDOFF §2b-3, "Two PRE-EXISTING engine bugs... item 2". The undo trigger, and the proof that it reaches exports and saves, are new. It is also in v1.2.0, so it is not a regression.
- **Fix:** make the branch unconditional (always clear the rows, then rebuild from Array.isArray(...) ? ... : []). HANDOFF asks for this to be fixed together with known bug #1 (stale closures after the re-key).
- **Needs a ruling:** no. Gate required.
- **Size:** small, about 10 lines plus a picker leg (load a file with custom phases, then one without).

M-3. A hiatus whose weeks the user blanked comes back as 2 weeks after Load, undo, crash recovery or a shareable copy

- **What happens:** clearing a hiatus row's Weeks box switches that hiatus off for the session, with no complaint. The file correctly saves weeks: "". On restore, addHiatusRow renders value="{prefillWeeks|2}", so the break re-arms at 2 weeks. Every phase crossing it moves 2 weeks later, silently.
- **Repro:**
 1. Phases tab ▶ All-phase hiatus. On the 12/21/26 row, delete the "2". The band vanishes.
 2. Save, then Load, or press Cmd+Z after any other edit. The break is back and Post ends 2 weeks later.

- **Evidence:** `SCHED_blankboot.json` (dist: the row comes back as weeks "2", and Post's last week is 3/1/27 instead of 2/15/27); `SCHED_core1.json` H1 (save and load) and H2 (undo).
- **Fix:** default only when the argument is absent (`prefillWeeks == null ? 2 : prefillWeeks`), and escape both prefills (which also closes an H-1 sink).
- **Needs a ruling:** no. Gate required.
- **Size:** one or two lines.

M-4. Day overrides move inconsistently when Production moves

- **What happens:** the same plan gets a different wrap depending on which tool moved it.
 - **Shift From** on a phase after Production moves **every** day override, even though Production stays put. Measured: wrap 10/27/26 became 10/28/26.
 - **Shift All** moves overrides by calendar days while holidays and locked hiatuses stay put, so a worked holiday stops matching its holiday.
 - A **month-view drag** of Production, **Rebuild From** and **Close all gaps** do not move overrides at all.
- **Repro:** load `dayoverrides.sptcal`. Shift From ▶ Post ▶ 1 wk Later. The Production start is unchanged, but the half, off and worked marks have all moved and the wrap changed.
- **Evidence:** `SHIFT_dist.json` D1/D2 (dist); `PROD_ov.json`; `SHIFT_stores.json` B1/B2; `MONTH_dragov.json` D1 (drag: wrap 10/27 → 11/3, "2 added" → "1 added"; control D2, Shift All, re-keys correctly); `SHIFT_rebuild.json` R1/R2.
- **Fix:**
 1. Shift From: re-key overrides only when Production itself moves. That is one line, and a clear bug.
 2. Drag, Rebuild and Close gaps: **your ruling first**. Do overrides travel with the shoot (by shoot-day index) or stay on calendar dates?
- **Needs a ruling:** yes for part 2. No frozen edit. Not the save format.
- **Size:** 1 line for Shift From; small-medium for the rest once ruled.

M-5. Snap to Mon and per-phase hiatus names carry into older files, and New keeps them

- **What happens:** `fields.byId` only writes the ids a file contains. A calendar saved before `snap-<key>` existed, opened after one with a phase's snap turned off, is scheduled **unsnapped**, with wrong dates. Per-phase hiatus names leak the same way. New and Reset All also keep the previous calendar's snap-off and hiatus names.
- **Repro:** untick Snap to Mon on Pre Prep, then Load `tests/fixtures/v1.0.0-saved.html`. Pre Prep reads 3/18/26 → instead of the correct "3/16/26 → ... Snapped to Mon".
- **Evidence:** `SAVE_carry.dist.json` C1 (dist) and C2 (a hiatus name from show A appears on the legacy file's grid); `SAVE_reset.json` R1.
- **Fix:** in `applyStateSnapshot` and `resetAll`, reset every `.phase-snap-cb` to checked and every `.phiatus-name` to empty when the file lacks the id. Use constants, not `defaultChecked`, which Export-shareable-copy can overwrite.
- **Needs a ruling:** no. Gate required.
- **Size:** small, about 4 lines.

M-6. No conflict detection: a stale tab or a colleague's copy silently overwrites the file

- **What happens:** Save (and autosave, every 10 minutes) writes back through the file handle without checking whether the file changed since it was loaded. Two windows on the same file, or two people on a shared or synced drive: the stale one wins, with no message on either side.

- **Repro:** open the same `.sptcal` in two windows. Save in window 2. Edit in window 1 and save. Window 2's work is gone.
- **Evidence:** `SAVE_stale.dist.json` S2 (`writes: 1, colleagueNoteSurvives: false`). Autosave doing the same is inferred from the code, not observed.
- **Fix:** record `lastModified` and size at load and after each write. Before writing, compare. Autosave suspends with a "file changed on disk" status, and Save asks.
- **Needs a ruling:** no. The design choice (ask, or refuse) is yours.
- **Size:** medium, about 40–60 lines.

M-7. Union-holiday data stops at 2030 in every region

- **What happens:** the holiday table covers 2026–2030 only. A shoot running into 2031 (or back into 2025) skips no holidays: the wrap comes out early, and the Holidays card lists nothing, with no notice. The data inside 2026–2030 is **correct**: an independent recomputation of all 15 regions found 0 missing and 0 extra.
- **Repro:** set Production to start in October 2030 with enough days to cross New Year. The 2031 holidays are not skipped.
- **Evidence:** `PROD_sched.json` `hol_2031` and `hol_2025`; `PROD_holverify.static.txt` (the rules recomputed from PROJECT-CONTEXT §5 and diffed against the table).
- **Fix:** regenerate with a wider year range (e.g. 2024–2036). Never hand-edit. Prove 2026–2030 is byte-identical. Add a notice when shoot days fall outside the covered years.
- **Needs a ruling:** not frozen. The exports change only for calendars outside 2026–2030. Gate required.
- **Size:** small.

M-8. The waterfall PDF never word-wraps a note: long notes are clipped at both ends

- **What happens:** the screen and Excel wrap a long note (and a note in a taller row) over several lines. The direct PDF writer draws it on **one centred line**, cut off at both ends. The waterfall PDF is the document people distribute.
- **Repro:** type a 200-character note, or drag its row taller. Export PDF. The note's first and last words are missing.
- **Evidence:** `GRID_pdfwrap.json` (probe): one text op holds the whole note; 2 notes are cut in the PDF, 0 on screen; Excel wraps the same notes. The writer is identical in `dist`.
- **Fix:** word-wrap inside `buildWaterfallPdf` with `ttfTextWidth`, using the same greedy rule as `wrapLineCount` and the row's line budget.
- **Needs a ruling: yes, frozen** (`buildWaterfallPdf`). The PDF output changes, and the gate's PDF baseline moves.
- **Size:** medium, about 25 lines.

M-9. Excel's header trimmer can cut through a formatting code

- **What happens:** once header lines carry formatting (bold, size, colour), trimming to Excel's 255-character limit can slice a `&"font"` code in half. The workbook header is then garbled, or Excel may offer to repair it. The trimmer can also drop the title and date while keeping the stats line.
- **Repro:** in Manual or Template mode, style several header lines and make them long, so the meter warns "about 541 of 255". Export Excel.
- **Evidence:** `GRID_hdrcut.json`: unterminated `&"font"` at 59. What real Excel then shows was not verified.
- **Fix:** keep each line as {code, text} until assembly, shave only text, and drop a line whose text reaches zero.
- **Needs a ruling: yes, frozen** (`exportExcel`).
- **Size:** small–medium, about 20 lines.

M-10. After a shift, an edited auto-note keeps the old start and wrap dates

- **What happens:** once a user adds a line to an auto-note ("Start Principal Photography 6/29/26"), the note is stored as literal text. After a shift it moves to the new week but still says 6/29/26, and it hides the correct auto-note. The Notes column then states the wrong start and wrap on screen, in Excel and in the PDF.
- **Repro:** add "Table read Thu" to the Start Principal Photography note, then Shift All 1 wk Later.
- **Evidence:** `SHIFT_autonote.json` N1 and N2; N3 (a worked holiday no longer inside the shoot after the shift).
- **Fix:** in `shiftCalendar`, swap the pre-shift auto text prefix for the post-shift one, or stop baking auto text into overrides.
- **Needs a ruling:** not frozen. It changes the note text that appears in the exports.
- **Size:** medium.

M-11. Rebuild From snaps a snap-off phase back to Monday

- **What happens:** Rebuild From ignores a phase's Snap-to-Mon setting and moves a Wednesday start back to Monday. That can pull Production's first shoot day earlier and move the wrap, with no message.
- **Repro:** give Production snap off and a Wednesday start. Rebuild from Production, working forwards.
- **Evidence:** `SHIFT_rebuild.json` R3 and R4; `SHIFT_dist.json` D3 (dist).
- **Fix:** read `snap-<key>` in the tools, and keep the typed day when snap is off.
- **Needs a ruling:** no.
- **Size:** small.

M-12. Shifts leave dragged row heights behind

- **What happens:** a row a user dragged taller for a long note stays on the old week after any shift. The note moves to a 20 px row and is cut off in the grid, Excel and the PDF, while an empty tall row sits where it was.
- **Repro:** drag a note's row taller, then Shift All 1 wk Later.
- **Evidence:** `GRID_shiftrow.json`; `SHIFT_stores.json` A5 (old week 60 px, new week 20.8 px).
- **Fix:** re-key `rowHeightsByWeek` in `shiftCalendar`, next to `noteFontSize`.
- **Needs a ruling:** no. Not frozen.
- **Size:** trivial, one line.

M-13. A waterfall note edit drops that week's month-only holiday labels from the month view and month PDF

- **What happens:**
 - Editing a week's note in the waterfall makes that week an override. From then on the month view shows only the override lines, so the week's holidays (July 4th, Thanksgiving) disappear from the month view and month PDF.
 - The reverse also happens: editing a waterfall note from the month view bakes the month-only holidays into the waterfall note and the Excel file.
- **Repro:** with the region US, add "Table Read" to the 6/29 week's note in the waterfall. In the month view, Independence Day (7/3 observed, 7/4) is gone.
- **Evidence:** `MONTH_wfnotes.json` W1 (lost: both Independence Day lines) and W3 (Thanksgiving baked into the waterfall cell). The one-digit-day date parsing the tracer suspected measured correct (W2).
- **Fix:** in the override branch, still emit the month-visible holiday auto-notes that are not already in the override.
- **Needs a ruling:** yes. It changes the month PDF, so it needs a `monthprint` A/B.
- **Size:** about 10 lines.

M-14. A phase whose last week straddles New Year adds an empty year block

- **What happens:** the export gains a blank year column and prints at about half size.
- **Repro:** end Post in the week of 12/30/30.
- **Evidence:** `SCHED_core2.json` Y1 (2031 block: 52 rows, 0 phase cells) and Y2.
- **Fix:** base the year-block end on the final week's Monday.
- **Needs a ruling:** not frozen. The exports change (only for these calendars).
- **Size:** one line.

M-15. Runaway counts freeze the tab and bloat saved files

- **What happens:** a named hiatus with a typo'd week count (e.g. 199,994) writes about 200,000 keys before the 600-week guard refuses the calendar. The snapshot grows to 8.4 MB, as do the crash backup and every undo step. Episode and block counts have no ceiling either: 8,000 rows get built before any guard.
- **Repro:** type 999999 into a named hiatus's Weeks box, or 8000 into Number of Episodes.
- **Evidence:** `SCHED_core1.json` H4 (`hiatusTextsKeys: 199994, snapshotBytes: 8403892`); `SCHED_episodes.json` E1.
- **Fix:** return early or clamp at `MAX_WEEKS` in the hiatus range, and cap episode and block counts (e.g. 200), setting `max` on the inputs.
- **Needs a ruling:** no.
- **Size:** small.

4. Low

Each entry gives what happens, the repro, the evidence, the fix, whether it needs a ruling, and a size.

- **L-1. A non-calendar JSON file is accepted as a calendar, and autosave then overwrites it.**
 - What happens: loading `package.json` or `{}` applies silently and makes that file the save target. The next autosave writes calendar JSON over it.
 - Repro: Load... ▶ All files ▶ any `.json` ▶ edit ▶ wait for autosave.
 - Evidence: `SAVE_foreign*.json` F1 (`autosaveWroteForeignFile: true`); `SAVE_parse.json` P1.
 - Fix: in `parseCalendarText`, require `fields.byId` to be an object; add the version gate from M-1.
 - Ruling: none (safe for every file ever written). Size: small.
- **L-2. Excel column widths above 255 and row heights above 409 pt.**
 - What happens: Single Column Mode on a calendar of about 4–5 years or more with a long note, or a very wide hand-drag, writes a width Excel's limit forbids. `check-xlsx.sh` flags `WIDTH > 255`.
 - Repro: Single Column Mode, a ~5-year calendar and a 60-character note ▶ Export Excel.
 - Evidence: `GRID_onecolwide.xlsx` (370.4), `GRID_xlsxbig.xlsx` (378.9), row 555.75 pt.
 - Fix: clamp in `sheetColumnWidths` / `exportExcel` / `installGridResizers`. Real Excel's reaction is unverified; Excel.app is installed if you want to check by hand.
 - Ruling: **frozen**. Size: tiny.
- **L-3. The month PDF can be cropped from Chrome's print dialog.** A custom Scale above 100%, or A4 paper on dense months. See §9.
 - Fix: the one-line `@page{size: letter landscape; margin:8mm}` pin.
 - Ruling: **frozen print CSS**. Size: 1 line plus a `monthprint` re-run.
- **L-4. ExcelJS loads from a CDN with no integrity check, and the page has no CSP or referrer policy.**

- What happens: `<script src=...exceljs@4.4.0...>` has no `integrity=`, blocks parsing, and gates boot on a network that silently drops packets. Offline Excel export fails **gracefully**, with the app's own "failed to load from the CDN" dialog.
- Evidence: `SUPPLY_cdnboot.json`; `SUPPLY_noexcel2.json`.
- A proposed CSP ran Excel, the waterfall PDF, Save, the month view and the update check with **zero violations** and no `unsafe-eval` (`SUPPLY_csp.json`). Only building a shareable copy timed out under it, so check that before adopting.
- Fix: add `integrity="sha384-Pqp51FUN2/qzfxZxBctF0stpc90NI6MYZpVqmo8m20SoaQCzf+arZvACKLkirlPz" crossorigin="anonymous" defer` (verify the hash out of band), plus the CSP meta and `<meta name="referrer" content="no-referrer">`.
- Ruling: none (`src/index.html` only; never the root legacy file). Size: small.
- **L-5. A U+FFFE character in a note makes the workbook invalid XML.**
 - Evidence: `GRID_xlsxctl.xlsx` (`check-xlsx.sh`: not well-formed at line 2, col 16097). C0 control characters and lone surrogates come out safely.
 - Fix: strip XML-illegal characters at note and title commit.
 - Ruling: none. Size: small.
- **L-6. Characters outside WinAnsi print as "?" in the waterfall PDF.**
 - What happens: CJK, emoji and arrows in notes or titles are corrupted in the distributed PDF only, with no warning.
 - Evidence: `GRID_xlsxuni.json`.
 - Fix: warn before export (non-frozen); the full fix is a font rework (frozen).
 - Size: small warning, or a large rework.
- **L-7. The Excel Date column is a formula chain with no cached values**, so phone and Quick Look previewers show blank dates.
 - Evidence: `GRID_xlsxuni.json` (103 of 103 uncached).
 - Fix: write `{formula, result}`.
 - Ruling: **frozen**, and every Excel baseline moves. Size: trivial.
- **L-8. Header-format values from a file are not type-checked.**
 - What happens: a bad size writes `&NaN` into the Excel header and `NaN Tf` into the PDF, or a size over 200 pt. An object where a line should be is stored as "[object Object]". A future-version `.spthdr` is refused.
 - Evidence: `GRID_spthdr.json`; `GRID_xsscal.json` pdf-nan-colour.
 - Fix: the H-1 validator covers it.
- **L-9. Header tokens resolve `Object.prototype` names**, e.g. `{constructor}` renders "function Object()...".
 - Evidence: `GRID_tokens.json`.
 - Fix: use `hasOwnProperty`, and precompute `lastIndexOf('')'` (the quadratic [case).
 - Size: trivial.
- **L-10. A hiatus date with a typo'd year blanks the whole calendar, and no field is marked.**
 - Evidence: `SCHED_core1.json` H3 (`ringedFields: []`).
 - Fix: extend `reflectStartDateValidity` to hiatus fields.
 - Ruling: none (chrome). Size: small.
- **L-11. A custom phase whose name starts with "Production" inflates the "N-Week Production Span" header.**
 - Evidence: `SCHED_custom.json` C1 (4 becomes 7).
 - Fix: count key `=== 'production'` only.

- Ruling: the exports change for such calendars. Size: trivial.
- **L-12. Decimal durations are truncated silently:** 2.5 weeks becomes 2, "1e3" becomes 1, and a 7.5-day episode becomes 7.
 - Evidence: `SCHED_core2.json` L1; `PROD_sched.json` `epdays_fraction`.
 - Fix: flag non-integers in the sidebar, or round up.
 - Size: small.
- **L-13. Anchor To "ends by" treats phases as ending Sunday**, so a Friday deadline costs a week.
 - Evidence: `SHIFT_anchor.json` AN1/AN2; `SHIFT_dist.json` D4.
 - Fix: use the phase's last weekday.
 - Size: small.
- **L-14. An undated note shifted onto an all-phase hiatus week disappears** from the waterfall and both exports.
 - Evidence: `SHIFT_stores.json` C1.
 - Fix: report it in the shift result, or merge it into the nearest visible week.
 - Size: small.
- **L-15. Rebuild From says "Nothing to place" after it has already moved a phase** and banked an undo step.
 - Evidence: `SHIFT_rebuild.json` R5.
 - Size: small.
- **L-16. Past 60 lanes, the month view draws new notes over lane 0.**
 - Evidence: `MONTH_print.MONTH_lanecap.json` P2/P5.
 - Fix: remove the cap. Ruling: **frozen** (`renderMonthView`). Size: 1 line.
- **L-17. When the month range shrinks below the month being viewed, the view jumps to the first month.**
 - Evidence: `MONTH_nav.json` V1 (April 2027 → January 2026).
 - Fix: clamp to the nearest month. Ruling: **frozen**. Size: 1 line.
- **L-18. An autosave tick commits and closes a note editor the user is typing in.**
 - Evidence: `SAVE_stale.dist.json` S3.
 - Fix: skip the tick while the editor is open.
 - Size: small.
- **L-19. A bad inline saved-state (a hand-edited shareable copy) throws out of boot**, leaving a page with no autosave or protection.
 - Evidence: `SAVE_inline.json` against the control.
 - Fix: share M-1's wrapper.
 - Size: small.
- **L-20. With no File System Access API (Safari, Firefox) there is no way to Load at all.** The file menu is hidden. This contradicts HANDOFF §3 B ("opening is unaffected").
 - Evidence: `SAVE_nofsa.json` (emulated by removing the API before boot).
 - Fix: an `<input type=file>` Load path with no id. At minimum, correct the doc.
 - Size: small-medium.
- **L-21. A shareable copy carries the sender's file-menu markup, with recent file names on a real profile, and their Grid Lines preference.**
 - Evidence: `SUPPLY_copyopen.json`.
 - Fix: H-2's fix, plus skipping `.prefs-card` in `reflectFieldsToAttributes`.
 - Size: small.

- **L-22. A crafted legacy .html freezes the tab through a quadratic regex**, about 86 s for 3 MB.
 - Evidence: `SUPPLY_redos.static.txt` (the ratio is 3.98 per doubling).
 - Fix: a bounded opener search, then `indexOf('</script>')`.
 - Ruling: the save-format reader, so the gate's restore leg must prove it. Size: small.
- **L-23. The Production region can change after month-view notes exist, and adding a custom holiday skips the recompute confirm.**
 - What happens: a region lock ignores day notes, so notes on the wrap day end up beside the wrong day.
 - Evidence: `PROD_region.json` `lock_ignores_daynotes` (the region changed to uk-london; wrap 10/20 → 10/19) and `custom_add_guard`.
 - Fix: include `dayNotes` in `hasNoteEdits`.
 - Size: small.
- **L-24. "Start after previous phase" chained off a snap-off phase overlaps it by two days** and adds a column.
 - Evidence: `SCHED_core2.json` P2.
 - Ruling: yes (chain to the next Monday, or keep the exact day?). Size: small.
- **L-25. A named hiatus's range loop runs before MAX_WEEKS can refuse it**, the root of M-15's freeze.
 - Evidence: `SCHED_core1.json` H4.
 - Fix: see M-15.
- **L-26. A one-day Friday Production shoot with snap off vanishes from the waterfall entirely.**
 - Evidence: `PROD_sched.json` `snapoff_fri1`.
 - Fix: H-3's fix covers it.

5. Notes

- **N-1.** A blank all-phase hiatus label prints "Hiatus" in the month view and month PDF but blank in the waterfall. A writer must copy whichever rule you choose.
- **N-2.** The runaway-duration message says "a typo in one of the years" even when the typo is a week count (`SCHED_core2.json` L2).
- **N-3.** While the calendar is refused (over 600 weeks), Export to Excel produced no file and no notice within 30 s (pane measurement).
- **N-4.** `#custom-hol-name` and `#custom-hol-date` (transient add-holiday inputs) are swept into `fields.byId`. Excluding them now would be a save-format change (gate 5). Probably leave it and document it.
- **N-5.** `tools/check-build.mjs` does not compare the engine's `APP_VERSION` with `version.json`. The two copies read 1.2.0 today.
- **N-6.** The app can be framed (clickjacking). The real exposure is low: file pickers need a real gesture, and storage is partitioned when framed.
- **N-7.** `deploy.yml` pins actions by tag, not SHA, and a manual dispatch can deploy any branch unless the environment restricts it to `main`.
- **N-8.** A refused file is still added to the recents list. `openRecentFile` clears `isDirty` only after awaiting IndexedDB. The legacy `header0overrides` migration uses the previous calendar's schedule. All static and all minor.
- **N-9.** Region caveats (Juneteenth for Local 52, Quebec's Good Friday / Easter Monday choice) never reach the UI.
- **N-10.** The harness trap: Chrome `--dump-dom` exits on a lone surrogate (§1).

6. Suspected: not reproduced

These are listed separately, as asked. None of them made the ranked list.

- **Autosave writing a mixed calendar by itself** (M-1). A real manual Save was proven. The autosave path is inferred from the code, because the headless harness never links a file.
- **What the H-1 script can do with live file grants:** writing to a user's files without a prompt once permission was granted this session. Reasoned from the API, not executed.
- **Shared origin:** the app unregisters **every** service worker on `greicher1.github.io`, including other repos'. The crash backup and file handles are readable by any page on that origin. This is certain from the code, but not executed, because the harness can't register a service worker.
- **Crash backup:** one slot per origin, so tab B's Save can delete tab A's recovery copy. A save within 3 s of an edit can leave a stale "Recover" prompt. Static only: IndexedDB never settles in headless Chrome.
- **Print dialog Margins:** "no effect" was a CDP artifact. The margin parameters never reached the layout, so the dialog's Margins setting is untested.
- **Leaving the print dialog open more than 60 s:** the safety-net cleanup may blank the preview if a setting is changed afterwards.
- **Pressing Cmd/Ctrl+P** instead of Export prints the whole app UI, because no print rule defends against it.
- **How real Excel reacts** to L-2, L-5 and M-9. `check-xlsx.sh` flags L-2 and L-5; Excel itself was not run.
- **One-digit-day note dates moving in the month view** (tracer MONTH-1): measured correct (W2).
- **A pill drag that returns to its origin** leaves snap cleared with no undo step (MONTH-9): not run.
- **Sim Post's "N weeks after Production begins"** counts hiatus weeks (SCHED-13): not run; a ruling question.
- **Mantine NumberInput clamp-on-blur desync** (typing 0 episodes shows 1 while the engine reads 0): needs real focus.
- `openRecentFile` requests **readwrite just to read**, which may cost a second permission prompt: not driven.

7. What was tested and found sound

- **Layout** at 768, 961, 1024, 1440 and 1920 px in the real pane:
 - 961 px and wider: no window scroll, the header on one row (63 px), the grid ending 20 px above the panel's bottom edge, nothing off the right edge;
 - 768 px: the stacked layout with vertical page scroll, as designed.
- **Preferences with localStorage blocked:** boots, renders, and the controls work (MONTH_nols on probe and dist).
- **Holiday data**, 2026–2030, all 15 regions: 0 missing, 0 extra against an independent recomputation of the rules.
- **The waterfall PDF with Grid Lines Solid and Dashed:** 0 rules through hiatus bands or spanned cells (GRID_pdfgrid_*).
- **Undo and redo** after every shift tool (8 of 8) and after the grid gestures tested (5 of 5).
- **Stale widths, spans and row heights:** keyed to columns that no longer exist, they leave both exports valid.
- **Bad files are refused cleanly:** empty, whitespace, truncated, malformed, array, scalar, `null` and `releases/v1.0.0.html`. A BOM before `{` is handled.
- **Offline Excel:** fails gracefully with the app's own notice, and the PDF still works.
- **A hostile version.json** renders as text.
- **The proposed CSP** works with no violations (L-4).
- **Crash recovery** works end to end in normal operation.

- **Scale:** a 540-week calendar paints in about 1 s; Excel in 75 ms, the waterfall PDF in 52 ms, the month switch in 76 ms.
- **The month PDF** prints one sheet per month on `monthscale` and on 5 edge fixtures: 6-row months, 60 notes in a week, one week, three years.
- **Excel formula injection** is not a practical vector: ExcelJS writes note and label text as string cells.
- **Typed UI text** is escaped everywhere tested.

8. Month PDF: how far the print dialog can break it

Measured on `monthscale.sptcal` (16 months, August 2026 in shrink-to-fit) through CDP `Page.printToPDF` against the real print document, with the pages rendered under Quartz:

Dialog choice	Result
Defaults (Letter, 100%, default margins)	16 sheets, one per month, whole
Background graphics OFF	identical: <code>print-color-adjust:exact</code> holds
Headers and footers ON	identical: no room in the 8 mm margin
Layout Portrait	identical: <code>size:landscape</code> wins
Scale 50% / 75%	whole, but tiny text or a blank band on the scale-mode month
Scale 125%	the scale-mode month loses its last two weeks (normal months whole)
Scale 200%	every month cropped (July loses weeks 4–5)
Paper A4 (the default outside the US and Canada; the app has UK, DE, AU and LT regions)	the scale-mode month's bottom row is clipped; normal months whole
Paper Legal / Tabloid	whole, with blank space
Margins None / 1 in	untested: the CDP margins never reached the layout (§6)

Contact sheets: `$SP/out/pd/png/contact-p7.png`, `contact-p8.png`, `contact-extra.png`. PDFs: `$SP/out/pd/`.

Recommendation: the direct month-PDF writer does NOT need to ship before department deployment. At default settings the print path produces a correct PDF, one sheet per month, and nothing a user can do in the dialog damages calendar data. The damage needs a non-default Scale, or A4 combined with a shrink-to-fit month, and it shows in Chrome's own preview before saving. Do two cheap things before deployment:

1. Pin the paper with `@page{ size: letter landscape; margin:8mm; }`. This is one line of frozen print CSS, so it needs your ruling. It matches the waterfall writer and the workbook. Save as PDF then hides the Paper option, and an A4 printer shrinks to fit rather than cropping. Prove it with the `monthprint` gate: headless already prints Letter, so it should be byte-identical.
2. Add one line to the help text: "print the month PDF at 100% scale". Nothing in CSS can stop a custom Scale. Only the writer can.

The writer then removes every row of the table above.

9. What a direct month-PDF writer must replicate

Compiled from `renderMonthView`, `exportMonthPdf` and the month and print CSS. Symbols are in brackets.

Page and pagination

1. One sheet per month: landscape, 8 mm margins, a 2 px inset inside the page box. The fit assumes Letter:
`PAGE_H=(8.5in-16mm)×96, PRINT_W=996`. [`exportMonthPdf; @page`]
2. The month range runs from the earliest segment start to the latest segment end – 1 day. An all-phase hiatus widens it only if it overlaps a segment. Day notes outside the range never print. The loop is capped at 240 months. [`monthRangeForSchedule`]
3. Weeks run Sun–Sat from the Sunday on or before the 1st. `weekCount = ceil((dow(1st)+days)/7)`, giving 4, 5 or 6 rows. [`renderMonthView`]
4. Spillover days carry full content (pills, notes, bands) on #FAFAF9, with the day number at opacity .35. [`.mv-out`]

Header

1. The header stack comes first, with no tools or arrows and margin-bottom 14 px. The purple Manual/Template box is stripped. [`.mv-header`]
2. The title bar: a baseline flex row with a 12 px gap. The left slot is at least 84 px, nowrap, 20/700, and reserves its space when empty. The title is flex:1, centred, 22/700. The date is 20/700 #000, nowrap. [`.mv-titlebar`]
3. The subtitle is on its own line, centred, 16/400, 2 px above. It is removed when empty. [`MV_NEW_SLOTS`]
4. The auto title is "<title> S<season> Full Prelim Production Calendar". The date is **local M.DD.YY** (not the waterfall's M.D.YY) and the same on every page. Manual is literal. Template goes through `resolveHeaderTemplate`. [`computeMvHeaderDefaults`]
5. Per-line formats:
 - size in px;
 - tri-state bold and italic;
 - colour;
 - alignment;
 - a highlight painted behind the text only (fit-content, 4 px side padding);
 - lines padded 0 3 px, with a minimum height of 14 px. [`headerFormatCss`]

Month bar, weekday row and grid

1. The month bar: "Month YYYY", 17/700, letter-spacing .01em, on #EDEDED, 7×10 px padding, with 2 px black top and side borders. [`.mv-monthbar`]
2. The weekday row: SUN...SAT, 11/700, uppercase, letter-spacing .04em, #726F68 on #F7F7F7, 6 px vertical padding, no internal dividers. [`.mv-dowrow`]
3. Every grid line is 2 px black: the frame, week separators and day dividers (none left of Sunday). In scale mode the horizontal lines are `round(2/scale)` px, and the frame's bottom line is drawn by the **last week**. [`--mv-line-y; .mv-scaled`]
4. Day cells: padding 3×5 px, day number 11/600 #333 at top left. Weekends on #F4F3F0.
5. Day-override marks:
 - off: 135° stripes #EFEFEF/#F8F8F8 at 4 px, number struck through at .5 opacity;
 - on: #FFF4E2 with the number in #7A5B14;
 - half: a "½" after the number, 700 weight, .75 opacity;
 - marked only where the simulation honoured the override. [`.mv-day-*`]

Lanes

1. The bar layer: 7 columns, 17 px lane tracks (`minmax(17px, auto)` in print), a 2 px row gap, padding 24 px top, 3 px sides, 14 px bottom, packed to the top. [`.mv-bars`]
2. Placement order per week row:
 1. all-phase hiatus bands;

2. phases sorted by start, each phase's pills followed by its own hiatus band;
3. Simultaneous Post;
4. per day: waterfall notes, then day notes.

Lanes are first-fit, top-down. Past 60 lanes today a block lands on lane 0 (L-16). [takeLane]

1. The lane count is $\max(4, \text{content}) + \min(\text{mvExtraLanes}[\text{key}], 6 - \text{base})$. The key is the Monday **before** the row's Sunday, a persisted off-by-one. [MV_MIN_LANES]

Pills and bands

1. Production covers exactly its shoot days (weekends, holidays and hiatus days are gaps). Other phases cover Mon–Fri within [start, end), excluding all-phase and own-hiatus days. Each contiguous run inside a Sun–Sat row is one pill. [pillRunsForWeek]
2. Pill style: 10/600, line-height 1.5, padding 1×8 px, margin 0 1 px, border 1 px rgba(0,0,0,.18), radius 9 px, nowrap with **ellipsis**. The fill is the phase colour. The text is the phase textColor, or black/white by luminance > 160. [.mv-pill; textColorFor]
3. The pill label is "<name without trailing wk N> Week N", where N counts Monday-weeks from the phase start. [segPillLabel]
4. Production's grey tag: 9/500, #5C6470, 7 px margin, same ellipsis.
 - Blocks mode: "Block N · Ep. 201, 202 / Block M · ...".
 - Episodes mode: "Ep. 201–203" (3+ consecutive numbers fold into a range).
 - None on a one-day run.
 - **Two per-user preferences (prefs.mvBlocks, prefs.mvEpisodes) suppress parts of it, so today's PDF depends on the viewer's localStorage.** [productionPillTag]
5. The half-day overlay: per half day, a layer with its bottom 50% at rgba(0,0,0,.34). Width $W=100/\text{span}\%$, at background-position $P=0/(100-W)\times 100$ (0 for a one-day pill). Episode pills get it too. [halfSlices]
6. The all-phase hiatus band:
 - Mon–Fri only, split at weekends, radius 9 px, centred text;
 - fill hiatusColors[Monday] or #FF0000, text by luminance;
 - label hiatusTexts[...] || 'Hiatus' (N-1). [hiRuns]
7. The per-phase hiatus band:
 - Mon–Fri inside both the phase and its hiatus, outside all-phase hiatuses;
 - label hiatusTexts['week|key'] or "<phase> Hiatus";
 - fill hiatusColors['week|key'] or #FF0000.
8. Simultaneous Post: looked up by the row's Monday, Mon–Fri non-hiatus days, #FFFF00 on #5B5B00, labelled "Simultaneous Post wk N". [simPostLabel]

Notes

1. Waterfall notes by day:
 - Auto notes sit on their own date (Monday if none), filtered by holidayView month visibility.
 - The label is "<label> M/D/YY".
 - Once a week is overridden, **only the override lines show** (M-13).
 - Each line goes to the day its trailing date names (one-digit days measured correct), else the pinned date, else Monday. [notesForWaterfallDate]
2. Day notes: dayNotes[iso] as a list (the legacy {text} shape is folded in). Empty entries are skipped. One block each. [dayNoteList]
3. Note block style:
 - square corners, 10/500 Inter, line-height 1.3, padding 2×4 px, pre-wrap, break-word, hyphens auto, clipped;

- `fill noteColors[week]` or the entry colour, default #7030A0.
- Note height in lanes is `mvNoteLineCount` at `mvNoteBoxWidth`: 1–3 lines, measured at the **live on-screen day-cell width** minus 2 (980 px fallback, 70 px minimum). In print the tracks then grow to the full wrapped text. **A writer must pick a fixed width, which will change today's window-dependent output** (MANTINE-SEAM §5.3).

Fitting to one sheet

- `reserve` = the first week's top minus the header's top, measured in screen media at `PRINT_W` (fallback 105 px).
- `reqH` per week = the bar layer's `scrollHeight`, at least 57. `niceH = 38 + max(maxLane, 4) × 19`.
- Fill mode (`reqTotal ≤ avail`): rows get flex-basis and min-height = `reqH` and grow by `max(0, niceH - reqH)`; if none wants to grow, all grow by `niceH`. `minmax(0, 1fr)` holds each week to its box.
- Scale mode: rows at `reqH`, body `scaleY((avail - 8) / reqTotal)` from the top left, minimum 0.05. Today that yields Type 3 glyph text in Chrome's PDF. `[MV_SCALE_HOLDBACK]`

Never printed, and fonts

- Never printed: "+", row-expand, the full-day overlay, hover chips, cursors, and the edit outline and hint. `data-ph` attributes travel into the print document but render nothing.
- Fonts: Inter for all month text (the embedded variable `woff2`, **not** Carlito), including the ½, · and – glyphs. Every fill prints (`print-color-adjust: exact`).

10. Completeness check: what got no test, and why

Phase 1 surfaces

- **The recents list, and reopening with a permission click.** It needs the native file picker and real `FileSystemFileHandles`, which cannot be driven from the pane or headless. IndexedDB never settles in headless Chrome. Crash recovery **was** tested end to end in the pane (§7).
- **Save As, and the first-save picker:** the native picker can't be driven. Save-to-handle was tested with recording fake handles, and one real Save click in the pane.
- **Two tabs:** simulated in one tab by changing the file between load and save (M-6). A true two-window run needs a person.
- **Real hit-testing of resize, span and swap handles, and of the day-override band:** the scripts dispatch events on the nodes, which proves the logic, not reachability. These gestures are already covered by the gate's `colswap/stint` legs and by `HANDOFF's elementsFromPoint` measurements.
- **Batch expand and pull back:** not scripted. The code read as sound, and it has existing coverage.
- **Blocks mode combined with the shift tools:** not exercised.
- **Month-view pill drag through real pointer events:** covered by the existing `monthdragundo` leg. The day-drag-turns-snap-off path was taken from the code (H-3/H-4).
- **The waterfall print-fallback PDF (`WF_PDF_MODE fallback`):** not tested. It is not the default path.
- **RTL text:** not specifically run. CJK and emoji were (L-6), and RTL takes the same `WinAnsi` path, so the same "?" result is expected.
- **The print dialog's Margins, a real printer, and the dialog left open more than 60 s:** these need a person at the dialog (§6).
- **Safari and Firefox:** noted, not tested, per the brief. L-20 and the unmeasured Safari print path are the known gaps.
- **Real Excel opening the flagged workbooks:** Excel.app is installed, but opening it needs someone at the GUI.

Phase 2 threats

- **Serving a service worker, and a real cross-origin framing test:** the harness serves everything from one origin as `text/html`. Both are reasoned from the code (§6, N-6).
- **Whether other pages are published under greicher1.github.io:** only this app's URL and its `version.json` were requested (read-only, as instructed). That decides how exploitable the shared origin is.
- **The month PDF under the proposed CSP:** headless print was not run with it.

Everything else in the brief got at least one executed test.

11. Verdict

NO-GO for department deployment as the build stands. GO once the must-fix list below lands and a full gate is green.

The reasons are:

- a security hole in the app's normal file-sharing workflow (H-1);
- a core export feature that is visibly broken (H-2);
- silently wrong dates, triggered by an ordinary month-view gesture (H-3, H-4);
- a load path that can overwrite the user's own file with a mix of two calendars (M-1).

Everything on the must-fix list is outside the frozen surface and changes no save-format key.

Must fix before deployment (together, roughly one working session plus one gate):

1. **H-1:** validate file-supplied values in `applyStateSnapshot` and the `.spthdr` reader. Not frozen. Small-medium.
2. **H-2:** empty the React hosts in `buildSavedHtml`'s clone. Not frozen. Small.
3. **H-3 + H-4:** the overlap week test and a Monday-anchored grid in `computeSchedule`. Not frozen, **but it changes the Excel and PDF output for snap-off calendars, so it needs your ruling**. Small-medium.
4. **M-1:** all-or-nothing restore, the version gate and the shape gate (this also fixes L-1 and L-19). Not frozen. Small-medium.
5. **M-2:** the unconditional custom-phase restore. Not frozen. About 10 lines.
6. **M-3:** `addHiatusRow`'s default only when the argument is absent. Not frozen. One line.

Strongly recommended before deployment (cheap):

- M-5 (snap and hiatus-name carry-over, about 4 lines);
- M-4 part 1 (Shift From, 1 line);
- M-7 (regenerate holidays to 2036);
- L-4 (the ExcelJS `integrity` attribute, plus the CSP and referrer meta);
- L-3 (the `@page` Letter pin; frozen, needs your ruling);
- M-12 (row heights follow shifts, 1 line).

After deployment: everything else, starting with M-6 (conflict detection), M-8 (PDF note wrapping, frozen), M-9 (Excel header trimmer, frozen), M-10, M-11 and M-13. Then the direct month-PDF writer, as you ruled.

Each fix happens only on your approval, one at a time, with the gate.